Work Integrated Learning Programmes
M.Tech Software Engineering



Scalable Services

Assignment 2

Containerization and Deployment of Microservices

Movie Ticket Booking System

By: Group 9

Name	BITS ID
Gujar Neel Lukesh	2025tm93111
Yamuna P Hegade	2025tm93019
Bhandari Akshay Sunil	2025tm93112
Naveen Venkat P	2025tm93528

1. Application Overview and Deployment Architecture

1.1 Application Overview

The Movie Ticket Booking System is a microservices-based application developed in Assignment 1 and containerized in Assignment 2. The system supports movie ticket booking through five independently deployable services.

Services:

- User Service
- Show Service
- Booking Service
- Payment Service
- Notification Service

The application follows the Database-per-Service pattern and uses REST APIs for inter-service communication. Booking Service acts as the Saga Orchestrator to coordinate booking, payment, and compensation workflows.

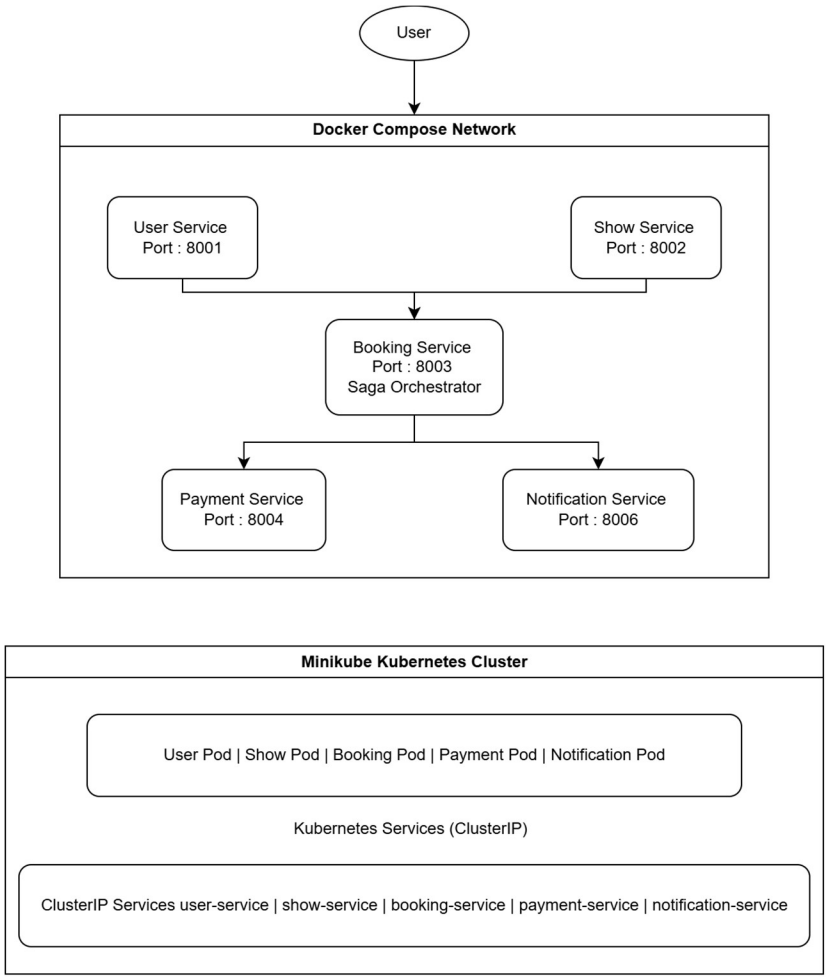


Figure 1. Containerization and deployment architecture of the Movie Ticket Booking System using Docker Compose and Kubernetes (Minikube).

2. Docker Containerization and Docker Compose Deployment

2.1 Containerization

Each microservice was containerized using a dedicated Dockerfile. Configuration settings such as service URLs were externalized through environment variables instead of hard-coded values. Database files, virtual environments, and temporary files were excluded using .dockerignore to create clean and reproducible images.

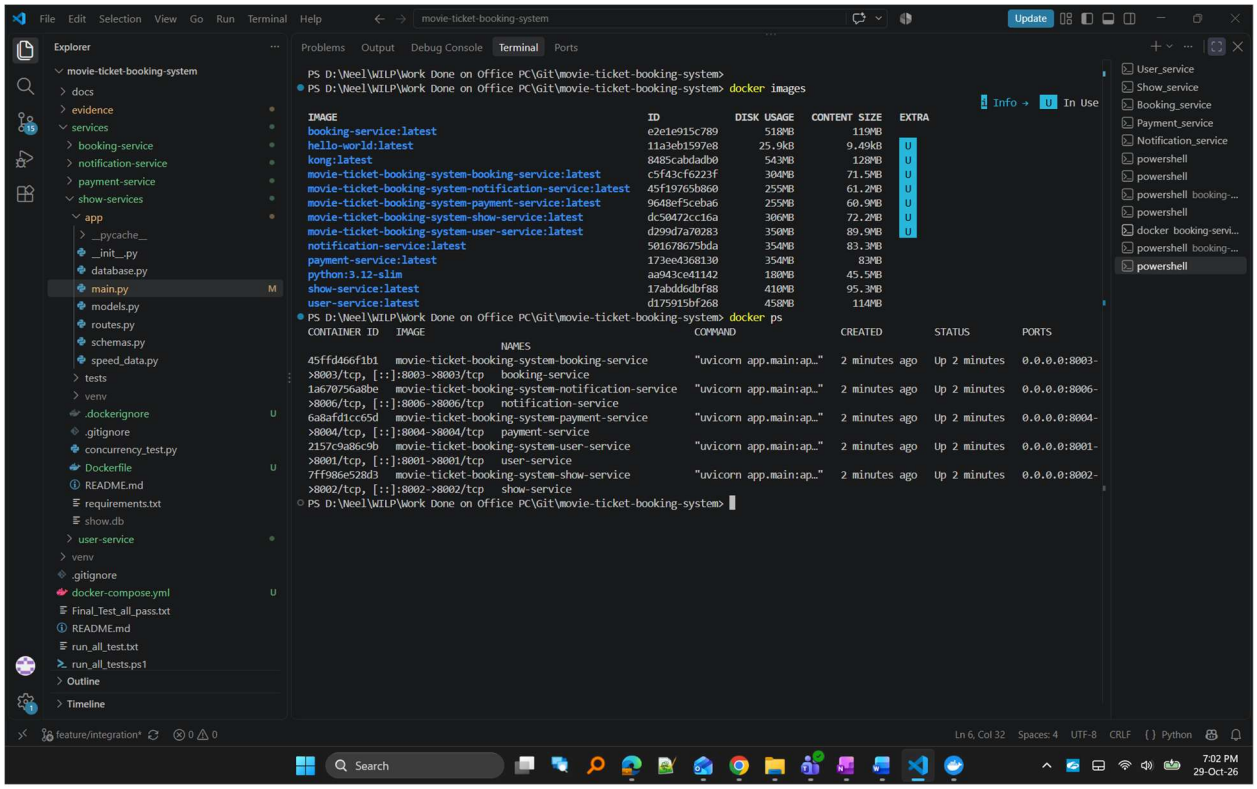


Figure 2. Docker images created for all microservices.

2.2 Docker Compose Deployment

The complete application was deployed using Docker Compose. A single command was used to build images, create containers, and configure service communication.

docker compose up –build

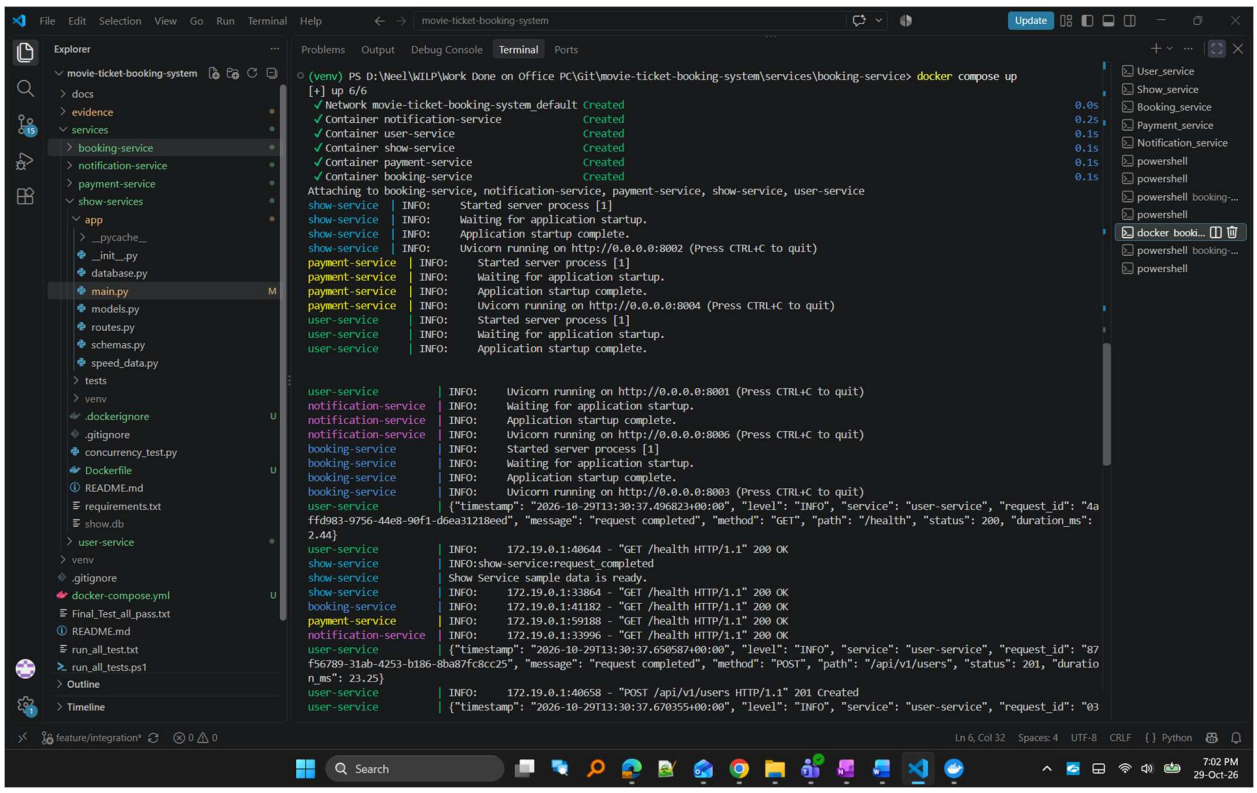


Figure 3. Successful deployment of all microservices using Docker Compose.

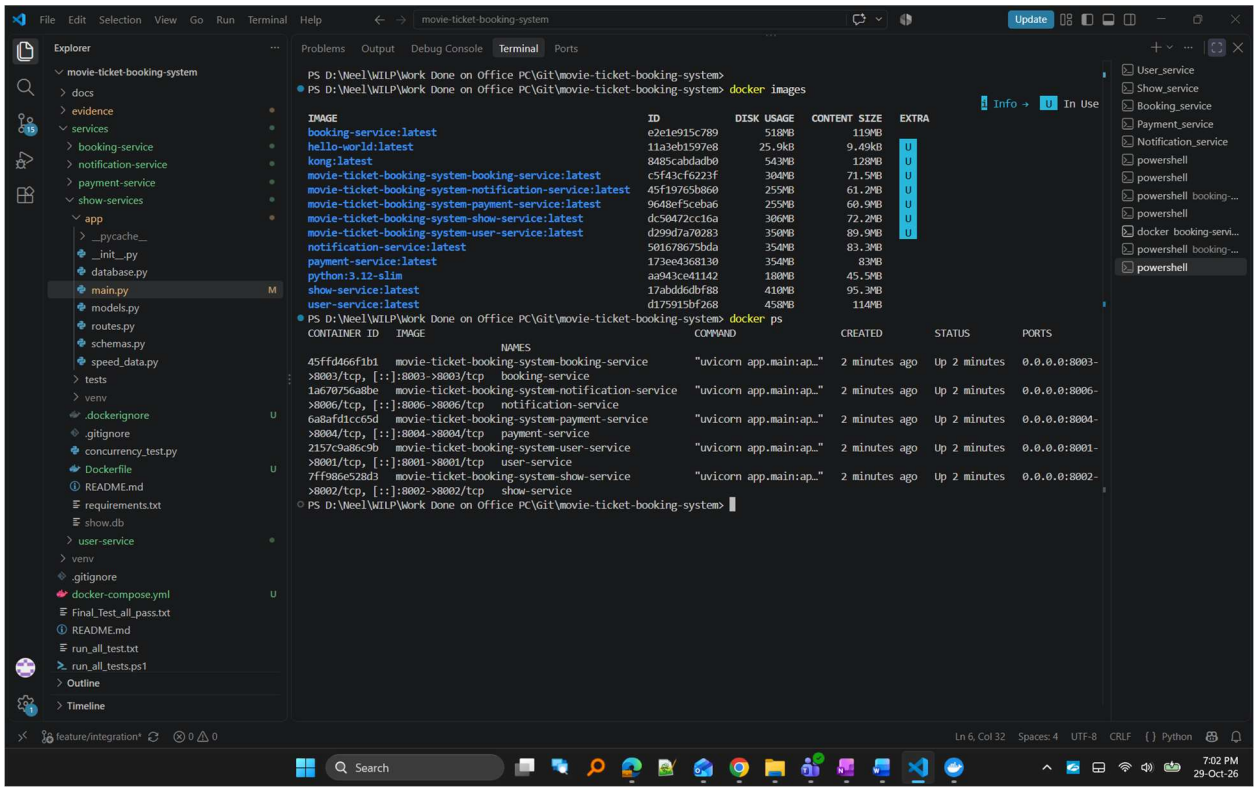


Figure 4. Running containers after Docker Compose deployment.

2.3 Main Workflow Validation

The deployed application was validated using the integration_demo.py script. The script performs health checks, booking validation, payment validation, Saga compensation validation, and notification verification.

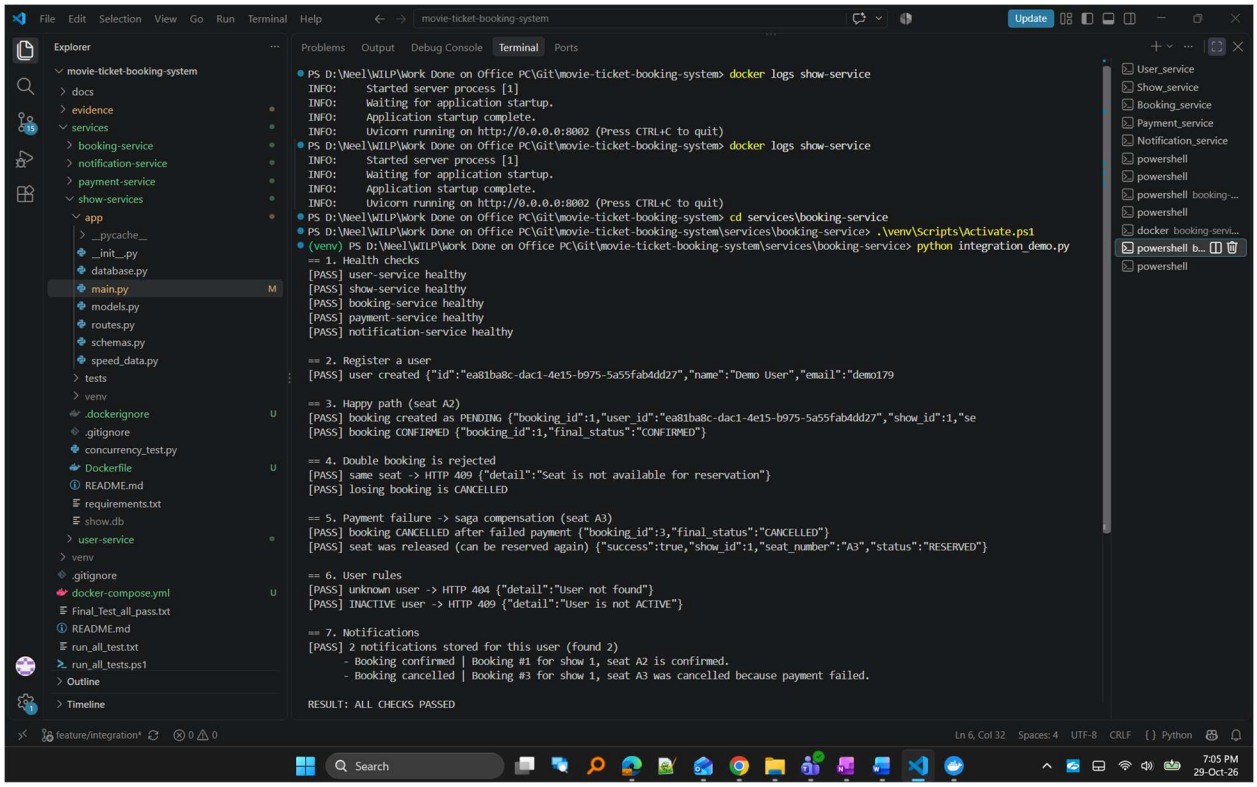


Figure 5. Successful end-to-end validation of the deployed application.

---

3. Situated Learning: Deployment Troubleshooting Challenge

3.1 Problem and Symptom

During Docker Compose deployment, the Booking Service container failed to start even though the application worked correctly during development.

Observed error:

> exec: "uvicorn": executable file not found in $PATH

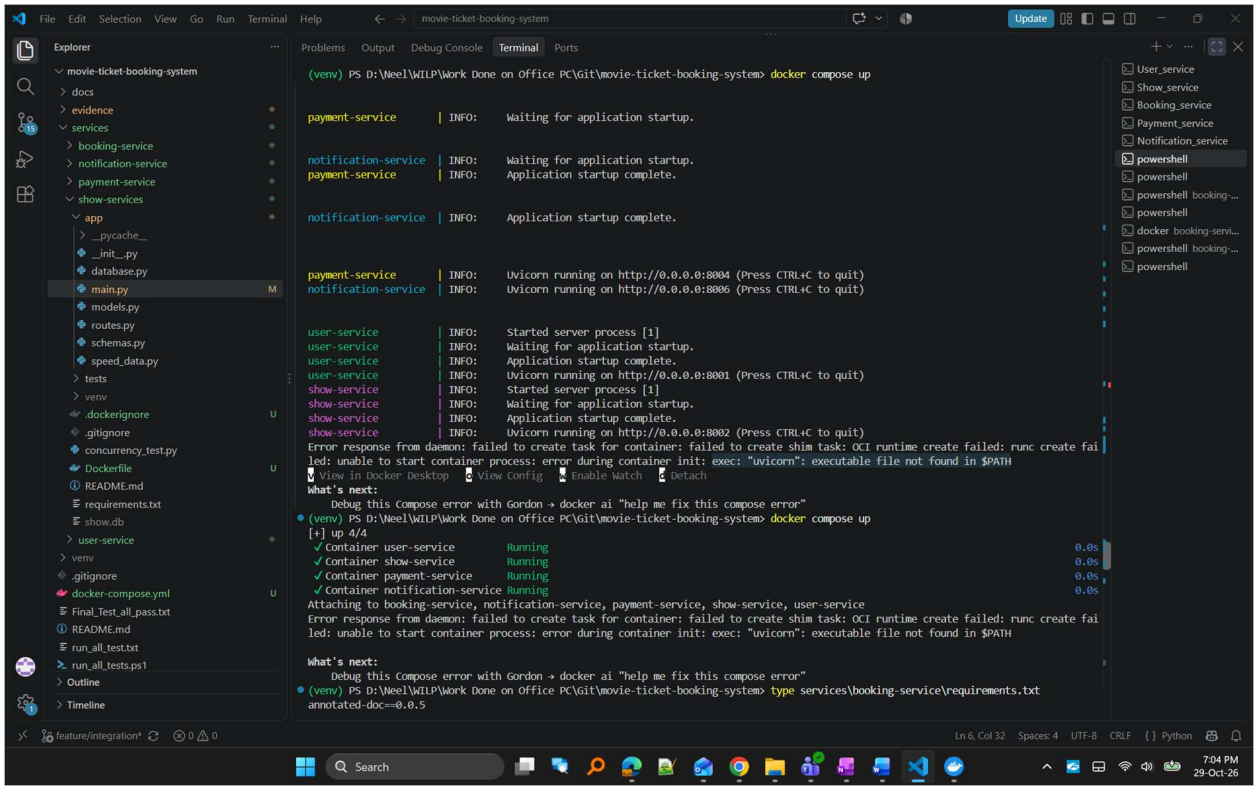


Figure 6. Booking Service startup failure encountered during container deployment.

3.2 Diagnosis

The issue was investigated using Docker deployment logs.

Commands used:

```
docker compose up

docker logs booking-service
```

Investigation revealed that Booking Service requirements.txt contained:

```
unicorn==2.1.4
```

instead of the required:

```
uvicorn
```

3.3 Solution and Validation

The incorrect dependency was removed and replaced with:

uvicorn[standard]>=0.30

The Booking Service image was rebuilt and redeployed successfully.

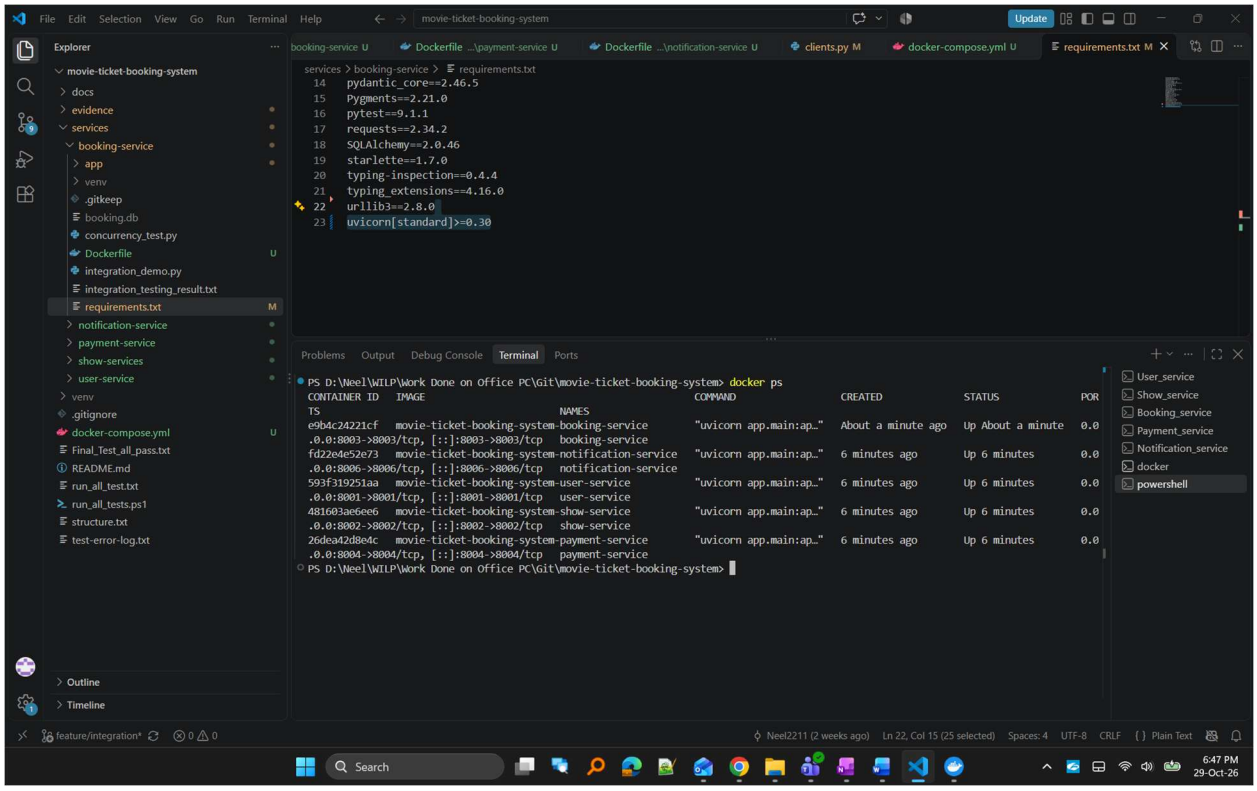


Figure 7. Corrected dependency configuration and successful container deployment after troubleshooting.

3.4 Learning Reflection

This issue demonstrated that containerized environments depend entirely on correctly declared dependencies. Local development environments may contain installed packages that mask configuration issues. Dockerized deployments help identify missing or incorrectly specified dependencies early and improve deployment reproducibility.

---

4. Advanced Task and Evidence

4.1 Kubernetes Deployment Using Minikube

The application was deployed on Minikube using Kubernetes Deployments and Services. Deployment manifests were created for all five microservices and executed using kubectl.

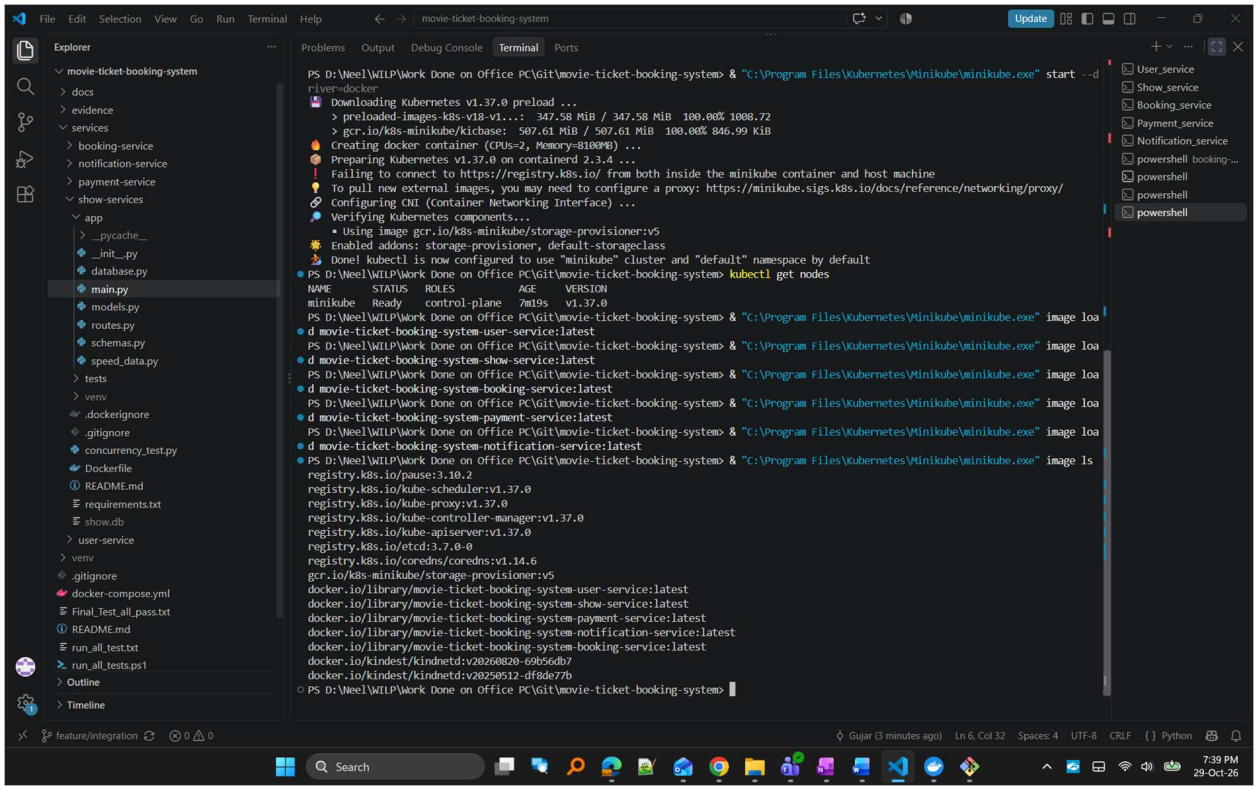


Figure 8. Minikube Kubernetes cluster in Ready state.

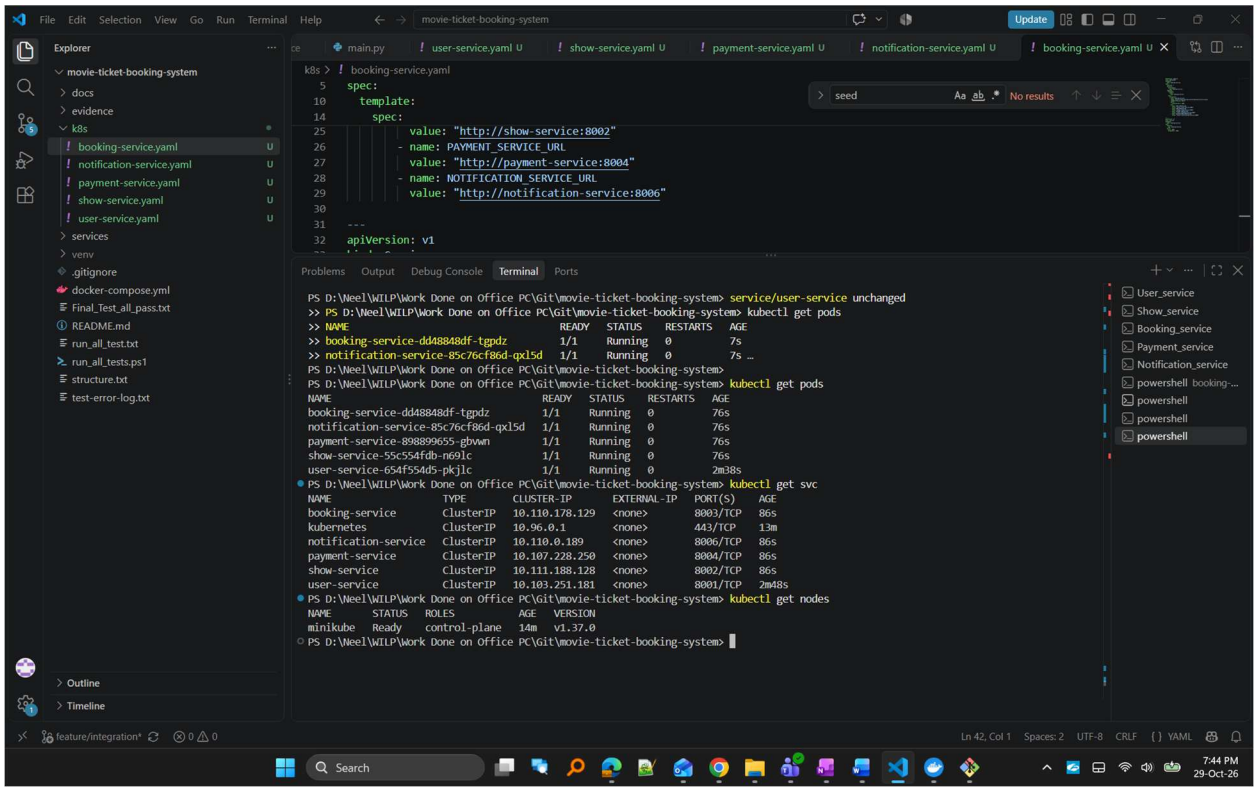


Figure 9. Kubernetes pods deployed for all microservices.

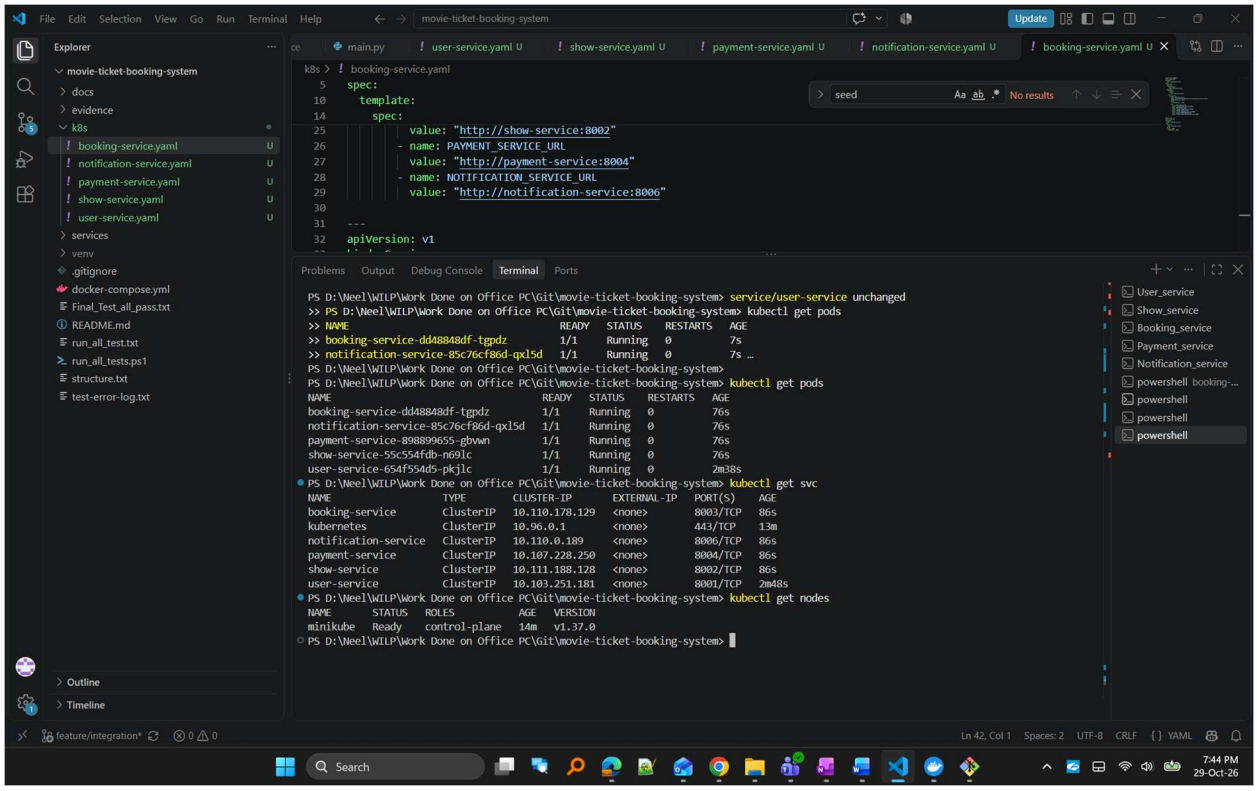


Figure 10. Kubernetes services exposing microservice endpoints.

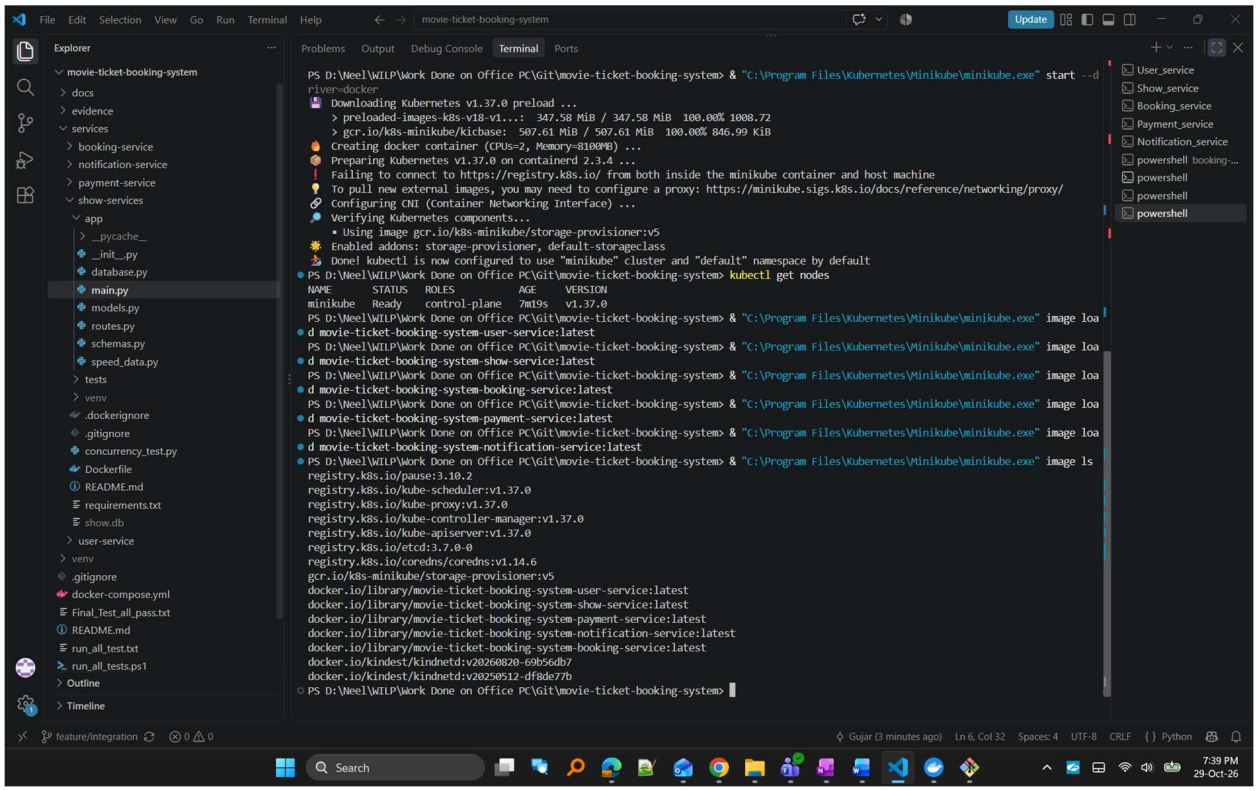


Figure 11. Application container images loaded into Minikube.

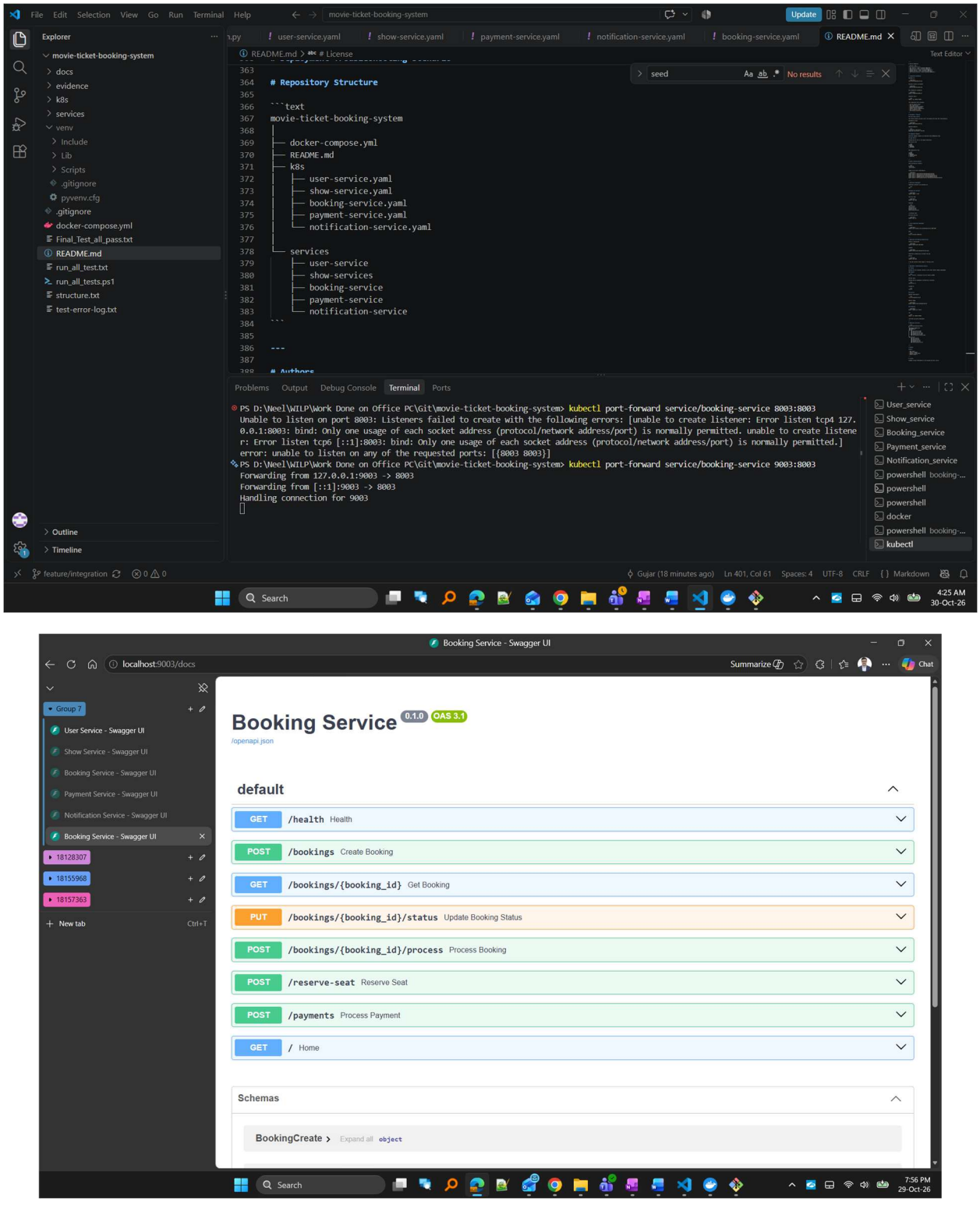


Figure 12. Booking Service API documentation accessed through the Kubernetes deployment using port forwarding.

This confirms that the Kubernetes-deployed Booking Service was reachable through a Kubernetes Service and successfully exposed using port forwarding, demonstrating that the application was functioning within the Minikube environment.

4.2 Kubernetes Self-Healing

Kubernetes self-healing was demonstrated by deleting a running Booking Service pod. Kubernetes automatically detected the missing pod and created a replacement instance without manual intervention. The Deployment controller continuously maintains the desired state of the application. When the Booking Service pod was manually deleted, Kubernetes automatically detected the deviation and launched a replacement pod without requiring manual intervention.

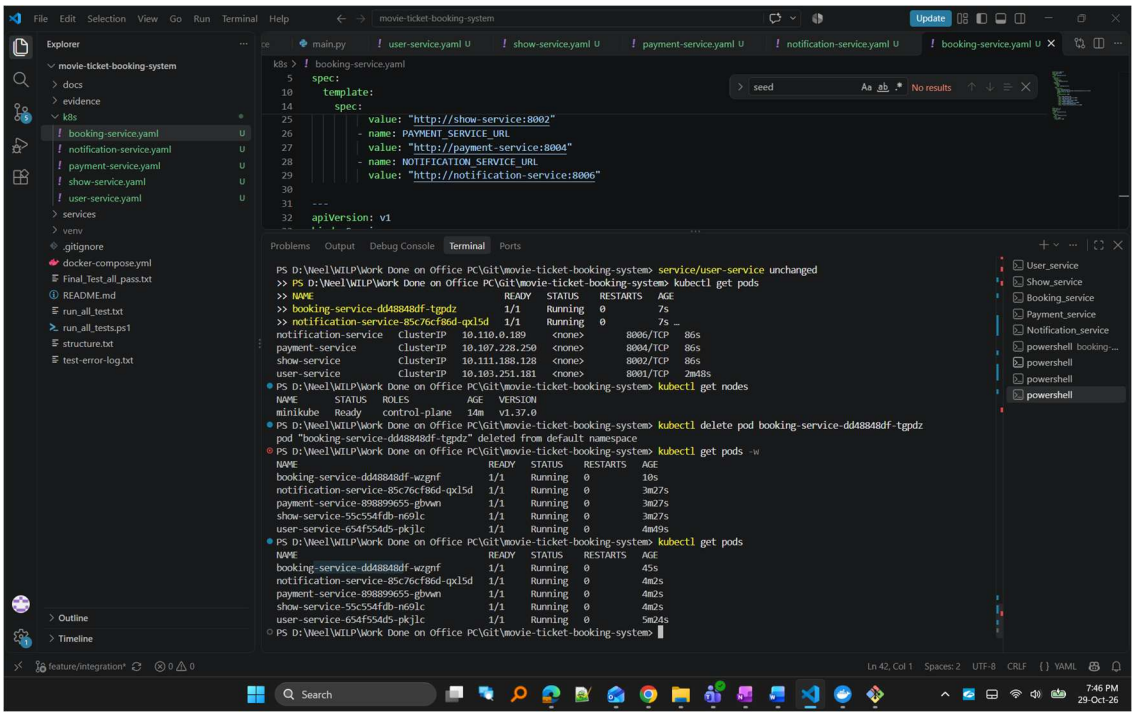


Figure 13. Kubernetes self-healing demonstrated through automatic pod recreation.

1. Pod deletion: pod “booking-service-dd48848df-tgpdz” deleted from default namespace
2. New pod automatically recreated: “booking-service-dd48848df-wzgnf” started automatically with a different pod name.

---

5. Individual Contribution Summary and AI Use Declaration

Contribution Summary

Member	Contribution
Neel Gujar	Dockerfiles, Docker Compose deployment, troubleshooting, Kubernetes deployment
Yamuna P Hegade	Testing, deployment validation, documentation
Bhandari Akshay Sunil	Containerization, integration testing, deployment support
Naveen Venkat P	Validation, Kubernetes testing, deployment verification

---

AI Use Declaration

Generative AI tools were used to assist with deployment guidance, troubleshooting support, documentation drafting, and report refinement. All implementation activities, testing, deployment validation, debugging, and final submissions were performed and verified by the team.

Appendix

GitHub Repository:

https://github.com/Neel2211/movie-ticket-booking-system.git

Demo Video:

https://drive.google.com/file/d/1SxZ1UUtDPXtZE4iufNK1sM1TTe1j_mF8/view?usp=sharing

Supporting Documents:

https://drive.google.com/drive/folders/1p0ZzCMmgtukiNlwRIEFjlvoasr9vUWKn?usp=sharing

---